Project Definition & Proof of Concept: Bayesian Belief-Tracking Battleship Agent

Student: Hyrum Lutz

Course: CS-4300 Artificial Intelligence

Private Repository: https://github.com/Rabbanah/CS-4300

Submitted Commit: `[to be commited]`

(I used Gemini to help make this document to match with the format of previous assignments)

1. Problem, Intended User, Intended Use, and Target Outcome

- **Problem:** Standard heuristic agents in partially observable search-and-destroy domains (such as *Battleship* solitaire) rely on greedy parity patterns and local neighbor checks that ignore global fleet constraints, sunk-ship identity feedback, and posterior probability distributions across remaining valid configurations.
- **Intended User / Client:** Game AI researchers, autonomous search-and-rescue simulation developers, and students benchmarking probabilistic inference against deterministic heuristics under strict action budgets.
- **Intended Use:** A controlled, reproducible simulation testbed where an autonomous agent locates and sinks a hidden multi-ship fleet on a 10×10 grid within a bounded shot budget.
- **Target Outcome:** Build and evaluate a sequential belief-tracking agent that combines exact constraint filtering, Monte Carlo / particle-filter belief estimation (to scale beyond exhaustive enumeration on full 10×10 boards), and information-theoretic target selection—achieving statistically significant reductions

in mean shots-to-victory and shot-budget failure rates compared to random and Hunt-and-Target baselines.

---

2. PEAS Assessment

PEAS Component	Specification
Performance Measure	1. **Completion Reliability:** Percentage of episodes where all ships are sunk within the shot budget (e.g., ≤ 75 shots on a 10×10 grid). 2. **Sample Efficiency:** Mean and median shots required to sink the entire fleet. 3. **Information Efficiency:** Average Shannon entropy reduction per probe. 4. **Computational Cost:** Wall-clock decision time per turn and particle/hypothesis survival rate.
Environment	A 10×10 discrete grid containing a hidden fleet of 5 ships (lengths 5, 4, 3, 3, 2; 17 occupied cells total) placed horizontally or vertically without overlap. The environment is **partially observable** (ship coordinates are hidden), **deterministic** (probe results are noise-free), **static** (ships do not move), **single-agent**, **sequential**, and **known** (grid dimensions and fleet manifest are known).
Actuators	A single discrete probing action `probe(row, col)` selecting an unprobed coordinate $(\mathrm{r}, \mathrm{c}) \in \{0..9\} \times \{0..9\}$.
Sensors	Structured `ShotResult(coord, hit, sunk_ship)` percepts indicating whether `(row, col)` was a `MISS` or `HIT`, and—when the final cell of a specific ship is hit—the identity of the newly `sunk_ship` (e.g., `"Destroyer"`).

---

3. Agent Decision Boundary, State, and Environment Interaction

```
+-------------------------------------------------------------------+
|                    Battleship Environment                         |
|  - Owns ground-truth 10x10 ship coordinates & shot counter        |
```



- **Decision Boundary:** The environment simulator owns the ground-truth fleet placement, enforces valid coordinate bounds, prevents duplicate shots, and determines terminal status. The agent never accesses hidden ship coordinates; it receives only `ShotResult` percepts after each probe.
- **Agent Internal State:** Sets of observed `hits`, `misses`, `sunk_ships`, and the active belief representation (a weighted set of consistent full-fleet placements).
- **Goal & Terminal Conditions:**
  - **Goal / Victory:** All 17 ship cells are probed (`is_won == True`) within the maximum shot budget $\mathrm{B}_{\max}$.
  - **Failure / Timeout:** Remaining shot budget reaches `0` before all ships are sunk, or an invalid action is issued.

---

4. Selected AI Strategy & Principal Technical Uncertainty

- **Selected AI Strategy: Sequential Bayesian Belief Tracking with Particle Filtering (Sequential Monte Carlo) and Information-Theoretic Selection.**
  - When the number of consistent hypotheses is small (e.g., after several hits or on smaller grids), the agent computes **exact marginal occupancy**

probabilities $\mathrm{P}\left(\mathrm{cell}_{\mathrm{r,c}}\ \mathrm{occupied} \mid \mathrm{history}\right)$.
    - On a full 10×10 grid with 5 ships, the unconstrained prior hypothesis space exceeds 10^{10} configurations. The agent therefore maintains a **Sequential Monte Carlo (SMC) particle set** of N valid, non-overlapping fleet placements consistent with all observations, rejuvenating depleted particles via MCMC / local ship resampling whenever observations prune too many hypotheses.
    - Target selection compares **Maximum Marginal Probability** ($\arg\max \mathrm{P}\left(\mathrm{occ}_{\mathrm{r,c}}\right)$) against **Maximum Expected Information Gain** (one-step lookahead Shannon entropy reduction).
- **Why It Fits:** Unlike local Hunt-and-Target rules, global fleet belief tracking automatically accounts for ship-length parity, tight packing constraints near board edges, and negative space left behind when specific ships are announced as sunk.
- **Principal Technical Uncertainty:**
    1. *Particle Deprivation on* 10×10 *Boards:* As hits accumulate late in an episode, naive rejection sampling struggles to generate valid fleet configurations that simultaneously satisfy all active hits and sunk-ship boundaries within tight per-turn runtime budgets.
    2. *PoC Resolution:* The Proof of Concept validates the exact Bayesian belief-update loop, sunk-ship constraint filtering, and probability-driven targeting on a tractable 6×6 grid before scaling to particle rejuvenation on 10×10.

---

5. Development & Evaluation Schedule

Milestone Window	Primary Deliverables & Engineering Tasks
Week 1 (Ending Nov 5)	Implement core `BattleshipEnv` (10×10 standard fleet, optional no-touching rule variant), deterministic seeded scenario generator, `RandomAgent` and `HuntAndTargetAgent` baselines, and unit test suite.
Week 2 (Ending Nov 12)	Implement full 10×10 `ParticleFilterAgent` with MCMC ship-resampling rejuvenation to prevent particle collapse after multiple hits, plus exact endgame solver fallback once hypothesis count drops below threshold.
Week 3 (Ending Nov 19)	Implement `InfoGainAgent` (expected entropy reduction policy), run controlled 1,000-episode paired benchmarks across all agents, and log component metrics to CSV.

Thanksgiving Week	No submission scheduled (buffer for edge-case debugging if needed).
Final Week (Ending Dec 3)	Statistical analysis (bootstrap confidence intervals, paired shot differences), failure-case trace analysis, final PDF report, and presentation slides.

---

6. Repeatable Evaluation Plan

All agents will be evaluated on identical paired suites generated from fixed random seeds (e.g., `1,000` distinct 10×10 fleet layouts with `--scenario-seed 4300`):

1. **Baselines vs. Proposed Agents:**
    - `random`: Uniform random unprobed selection.
    - `hunt_target`: Parity-based checkerboard search switching to orthogonal neighbor hunting upon a hit.
    - `smc_max_prob`: Sequential Monte Carlo belief agent selecting $\arg\max \mathrm{P}\left(\mathrm{occ}_{\mathrm{r,c}}\right)$.
    - `smc_info_gain`: Sequential Monte Carlo belief agent selecting maximum expected Shannon entropy reduction.
2. **Numerical Measures:**
    - **Solution Quality / Sample Efficiency:** Mean and median shots to sink all 5 ships (with 95% bootstrap confidence intervals).
    - **Reliability:** Win rate within strict shot budgets ($\mathrm{B}_{\max} \in \{60, 75, 100\}$) and zero invalid/duplicate probe rate.
    - **Computational Cost:** Mean decision latency (milliseconds per shot) and mean active particle count.

---

7. Expected Limitations, Failure Conditions, and Deployment Criteria

- **Expected Limitations:**
    - Assumes the fleet manifest (exact count and lengths of ships) is known a priori.
    - One-step entropy lookahead increases per-turn CPU time compared to simple marginal probability lookup.
- **Failure Conditions:**

    - *Particle Collapse:* If all N particles become inconsistent with a newly observed hit and MCMC resampling fails to find a valid fleet within the step timeout, the agent would fail to produce a valid posterior.
    - *Adversarial / Clustered Layouts:* Worst-case fleet arrangements tucked into corners can push shot counts near the budget ceiling.
- **Recommendation Criteria:**
    - **Deploy:** Achieves $\geq 99.5\%$ completion within 75 shots, reduces mean shots by $\geq 15\%$ over `hunt_target` with non-overlapping 95% CIs, and maintains $< 50\ \mathrm{ms}$ average latency per shot.
    - **Revise:** Outperforms baselines in shot efficiency, but suffers occasional particle collapse ($> 0.5\%$ failure rate) or exceeds latency targets, requiring improved MCMC proposal moves.
    - **Do Not Use:** Fails to statistically beat `hunt_target` on standard layouts or exhibits unrecoverable belief collapse on valid fleets.

---

8. Proof of Concept: Run Instructions, Evidence, and Feasibility Assessment

Run Instructions

From the `a10` project directory containing `poc_battleship.py` (Python 3.10+, standard library only):

```
python3 .\poc_battleship.py
```

Retained Execution Trace (`seed=4300`)

```
PS C:\Users\hlutz\Documents\CS-4300\a10> python3 .\poc_battleship.py
=== Battleship Bayesian Belief PoC (6x6, seed=4300) ===
Initial consistent fleet hypotheses: 2472

Step 01 | Probe (2, 2) | P(occ)=0.184 | H=0.690 bits | Result=MISS              | Remainir
Step 02 | Probe (3, 3) | P(occ)=0.206 | H=0.734 bits | Result=MISS              | Remainir
Step 03 | Probe (1, 1) | P(occ)=0.185 | H=0.691 bits | Result=MISS              | Remainir
Step 04 | Probe (4, 4) | P(occ)=0.202 | H=0.727 bits | Result=MISS              | Remainir
Step 05 | Probe (1, 4) | P(occ)=0.223 | H=0.766 bits | Result=MISS              | Remainir
Step 06 | Probe (4, 1) | P(occ)=0.248 | H=0.807 bits | Result=MISS              | Remainir
```

```
Step 07 | Probe (0, 3) | P(occ)=0.276 | H=0.850 bits | Result=MISS                | Remainir
Step 08 | Probe (2, 5) | P(occ)=0.327 | H=0.912 bits | Result=MISS                | Remainir
Step 09 | Probe (3, 0) | P(occ)=0.405 | H=0.974 bits | Result=MISS                | Remainir
Step 10 | Probe (5, 2) | P(occ)=0.545 | H=0.994 bits | Result=MISS                | Remainir
Step 11 | Probe (0, 0) | P(occ)=0.550 | H=0.993 bits | Result=MISS                | Remainir
Step 12 | Probe (5, 5) | P(occ)=1.000 | H=0.000 bits | Result=HIT                 | Remaining
Step 13 | Probe (3, 5) | P(occ)=0.556 | H=0.991 bits | Result=HIT                 | Remaining
Step 14 | Probe (4, 5) | P(occ)=0.950 | H=0.286 bits | Result=HIT (SUNK Cruiser!) | Rer
Step 15 | Probe (0, 2) | P(occ)=0.111 | H=0.503 bits | Result=MISS                | Remainir
Step 16 | Probe (0, 5) | P(occ)=0.125 | H=0.544 bits | Result=MISS                | Remainir
Step 17 | Probe (1, 3) | P(occ)=0.143 | H=0.592 bits | Result=MISS                | Remainir
Step 18 | Probe (2, 0) | P(occ)=0.167 | H=0.650 bits | Result=MISS                | Remainir
Step 19 | Probe (2, 4) | P(occ)=0.200 | H=0.722 bits | Result=HIT                 | Remaining
Step 20 | Probe (2, 3) | P(occ)=0.500 | H=1.000 bits | Result=HIT (SUNK Destroyer!) | R

Fleet destroyed in 20 shots (out of 36 cells).
```

Feasibility Assessment

- **Feasibility Confirmed:** The Proof of Concept validates that sequential Bayesian belief filtering over valid fleet placements solves the partially observable search problem without hardcoded parity or hunting heuristics. Notably:
    1. During Steps 01–11, maximizing marginal occupancy probability $\mathrm{P}(\mathrm{occ})$ naturally induces optimal diagonal coverage, pruning the hypothesis space from `2,472` down to `36`.
    2. By Step 12, deductive constraint propagation drives $\mathrm{P}(\mathrm{occ})$ at `(5, 5)` to **`1.000`** ($\mathrm{H} = 0.000$ bits) prior to probing, proving that the probabilistic estimator seamlessly captures deterministic logical entailment.
    3. Sinking the Cruiser at Step 14 and hitting the Destroyer at Step 19 narrows the remaining hypotheses down to `2`, allowing the agent to sink the final ship at Step 20 and converge to the single ground-truth fleet (`Remaining Hypotheses=1`).
- **What Must Change Next:** While exact enumeration across `2,472` hypotheses runs instantaneously on a 6×6 two-ship grid, exhaustive enumeration is intractable for a full 10×10 five-ship fleet ($> 10^{10}$ prior configurations). In the next development phase, we will replace exhaustive enumeration with a **Sequential Monte Carlo (SMC) particle filter** paired with **local MCMC ship-perturbation rejuvenation** so the belief distribution remains populated and well-mixed across all 17 hits of a full-scale match, switching back to exact enumeration only when the pruned hypothesis space drops below a tractable threshold.

---

9. References & AI-Assistance Disclosure

References

- Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson. Chapter 13 (Quantifying Uncertainty) & Chapter 14 (Probabilistic Reasoning over Time).
- Berry, N. (2011). *DataGenetics: Battleship Analysis* (Bayesian posterior occupancy grids vs. Hunt-and-Target parity strategies).

AI-Assistance Disclosure

- **Tool:** Gemini (Google DeepMind)
- **Material Effect:** Assisted in structuring the PEAS assessment, decision-boundary architecture, evaluation metrics, and implementing the Python 6×6 Bayesian belief-tracking Proof of Concept (`poc_battleship.py`).
- **Verification:** Executed `python3 .\poc_battleship.py` locally with `seed=4300` and verified the exact posterior probability calculations, hypothesis counts, and 20-shot completion trace.